



GEPA: Optimización Automática de Prompts con Reflexión

Cómo un LLM 'profesor' mejora automáticamente tus prompts mediante análisis de errores

DEMO TÉCNICA

REFLEXIO LAB

El Desafío: Optimizar Prompts es Lento y Manual

Antes de GEPA

- Escribes un prompt genérico basado en intuición
- Pruebas manualmente con casos aislados
- Ves errores, ajustas por prueba y error
- Repites durante horas para mejoras marginales
- Resultado depende completamente de tu experiencia

El proceso es artesanal, subjetivo y no escalable.

Con GEPA

- Defines prompt inicial y casos de prueba
- GEPA ejecuta y analiza errores automáticamente
- LLM "profesor" genera variantes optimizadas
- Búsqueda evolutiva identifica el mejor candidato
- Resultado: prompt especializado en minutos

El proceso es sistemático, medible y reproducible.

Qué Hace GEPA: Test-Driven Development para Prompts

01

Ejecutar

El prompt actual procesa micro-lotes de 3 casos para observar errores y aciertos. Evaluación ágil que permite iteración ultra-rápida sin consumir todo el dataset.

02

Reflexionar

El LLM Profesor analiza los errores detectados, razona sobre las causas raíz y propone mejoras específicas. Ejemplo: "El prompt olvidó mencionar límites monetarios, debería enfatizar precisión numérica".

03

Evolucionar

Genera variantes mejoradas del prompt, las evalúa sistemáticamente y mantiene las mejores. Se repite hasta alcanzar el score objetivo definido.

- 📄 Búsqueda evolutiva Pareto-eficiente: mantiene candidatos complementarios, no solo el mejor

TDD Clásico vs. Flujo GEPA

La analogía perfecta entre desarrollo de software y prompts

Fase TDD (Software)	Fase GEPA (Prompt Engineering)
1. Write Test	Definir Dataset de Entrenamiento y Validación (Inputs + Salidas Esperadas)
2. Red (Falla)	Baseline Run (Prompt inicial con baja precisión, ej: 40%)
3. Code & Refactor	Optimización Automática (Modelo Profesor reescribe basándose en errores)
4. Green (Pasa)	Prompt Optimizado (Alcanza métrica objetivo, ej: >90%)
5. Integration Test	Prueba de Robustez (Validar generalización en Test Set)

El Superpoder

En TDD tradicional, **tú** escribes el código. En GEPA, **la IA** analiza el error y reescribe el prompt. Es **TDD Autónomo**.

Resultados Reales: Impacto Medido en el Laboratorio

Mejoras consistentes con estrategia estudiante GPT-4.1-mini

Caso de Uso	Mejora	Detalle
Email Urgency	+35%	59.8% → 95%
CV Extraction	+21%	60.0% → 81.5%
Text-to-SQL	+33%	33% → 66%
RAG Optimization	+42%	52% → 94%

Mejoras consistentes de +21% a +42% en todos los casos sin cambiar de modelo, solo optimizando el prompt.



ROI: Inversión Mínima, Retorno Masivo

Análisis de costo-beneficio con datos reales

\$0.12

Costo Total de Optimización

Inversión única: 510 llamadas Task Model (\$0.04) + 25 llamadas Reflection Model (\$0.08)

106

Punto de Equilibrio

Llamadas necesarias para recuperar la inversión inicial

91,950%

ROI a 100K llamadas

Ahorro de \$109.88 sobre inversión de \$0.12

Ejemplo: 1,000 Llamadas

- 1. Sin GEPA: Modelo potente (GPT-4o) = \$1.25
- 2. Con GEPA: Optimización (\$0.12) + Mini (\$0.13) = \$0.25
- 3. Ahorro: \$1 USD
- 4. ROI: 833% de retorno

Escalabilidad del Ahorro

- 500 llamadas: ROI 360%
- 5,000 llamadas: ROI 4,502%
- 50,000 llamadas: ROI 45,925%

A mayor escala, mayor ahorro proporcional.

Estrategia Profesor-Estudiente

El secreto: desacoplar inteligencia de inferencia

- 1 Fase 1: Optimización**
Modelo PROFESOR (GPT-4o): Alta capacidad analítica para analizar errores y redactar prompt maestro. Inversión inicial controlada que se realiza una sola vez.
- 2 Resultado Intermedio**
Se genera un **Prompt Especializado** que captura la inteligencia del modelo profesor en forma de instrucciones precisas.
- 3 Fase 2: Producción**
Modelo EFICIENTE (GPT-4.1-mini): Ejecuta miles de llamadas con el prompt optimizado. Calidad de nivel GPT-4o al costo de GPT-4.1-mini.

Reducción del 90% en costos de producción

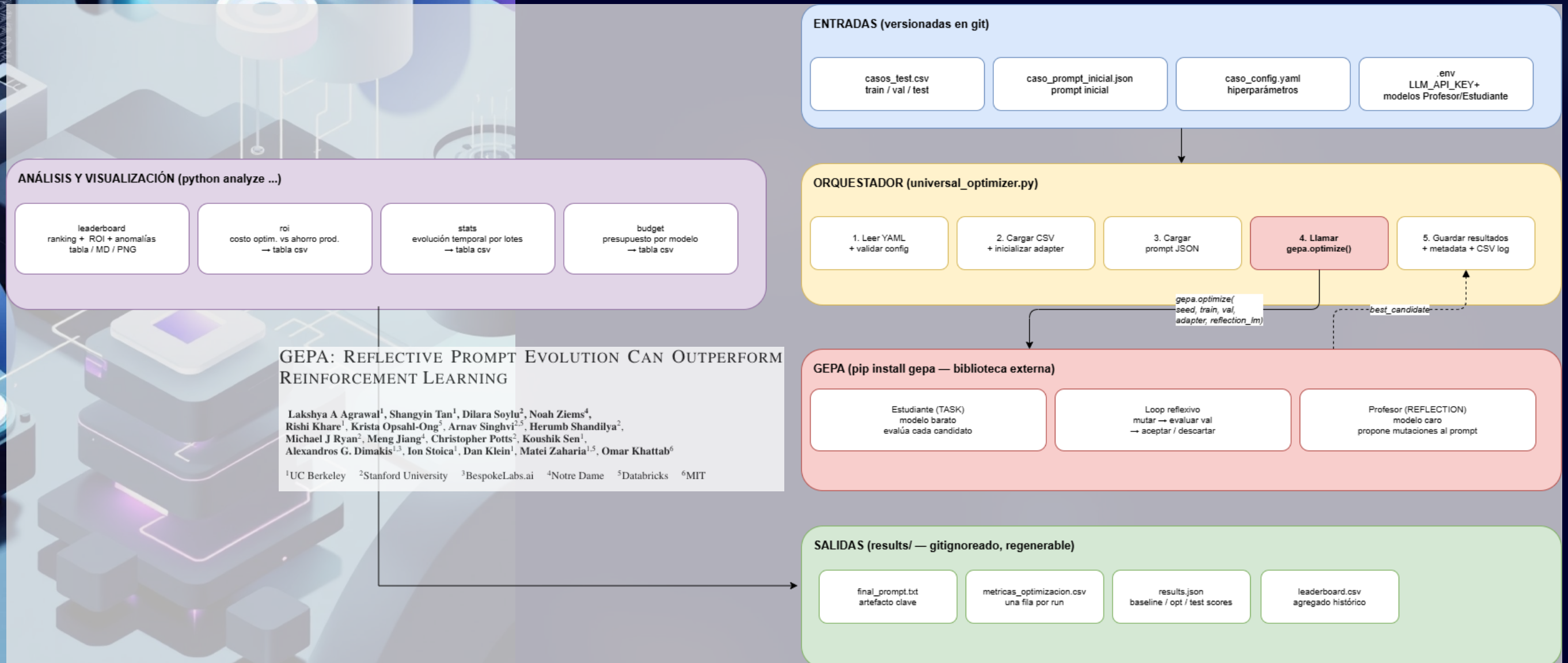
Inteligencia de GPT-4o al costo de GPT-4.1-mini

Learning Model



Arquitectura Reflexio

Componentes y flujo de datos



Para que usos es de valor?

- 1 Optimización de prompts
Un prompt puede ser verificado en su efectividad y optimizado
- 2 ROI alto para casos de uso frecuente
Un Modelo professor para optimizar y otro estudiante para producción
- 3 Aislo mi sistema del drift y cambios de modelo
Doy una medida profesional de exactitud semántica y verifico si con el tiempo hay desvíos
Si cambio el modelo base puedo reestablecer mi estado gral rápidamente
- 4 Calibrar mi Sistema con IA
Si mi Sistema esta sobrecalificado puedo elegir modelos mas simples y económicos

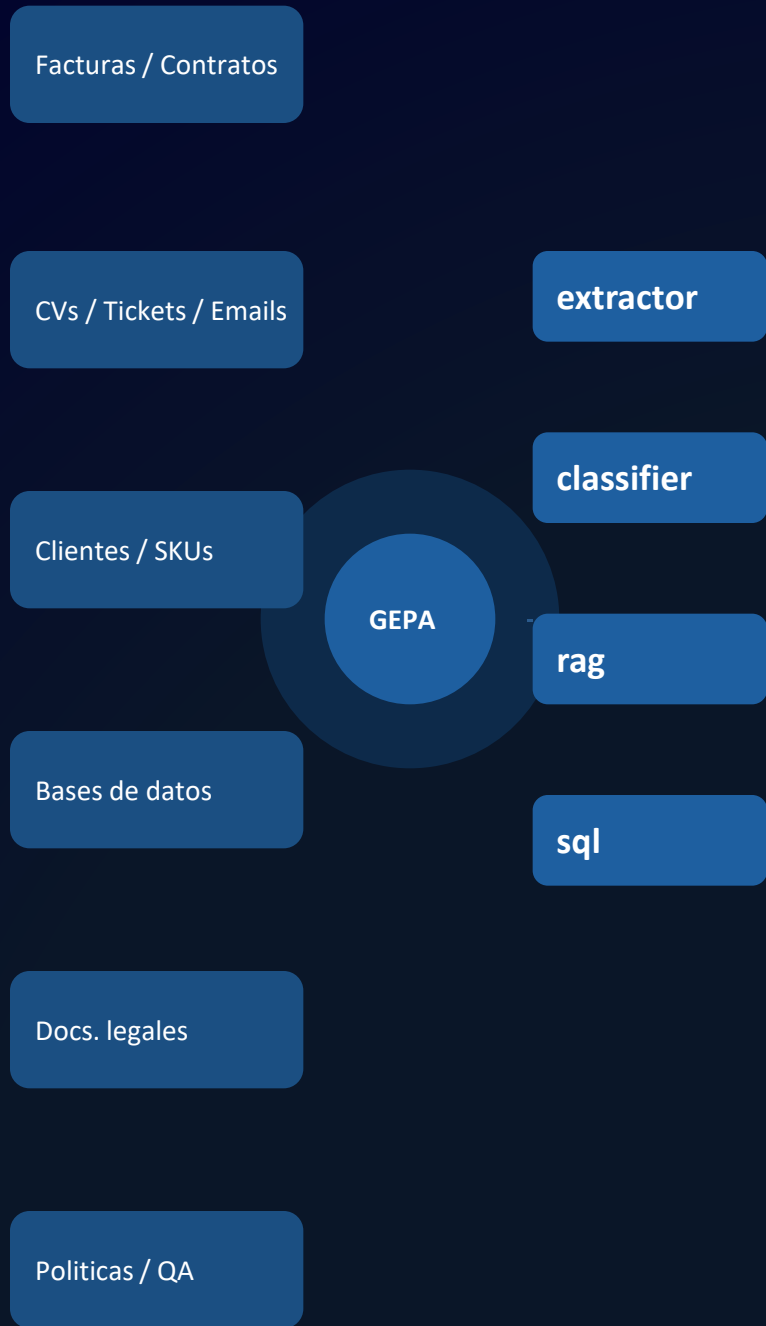
Learning Model



¿Cuándo aplicar GEPA?

Dominios y casos de uso por tipo de dominio

Dominio	Ejemplos
Extracción estructurada	facturas, contratos, historias clínicas, pólizas, expedientes legales
Clasificación / triage de alto volumen	CVs, tickets de soporte, emails (spam/lead/queja), moderación, fraude
Normalización y matching	deduplicación clientes/productos, mapeo a catálogos, direcciones sucias
RAG / Q&A sobre conocimiento propio	asistentes de soporte, búsqueda legal con citas
NL → SQL	analistas no técnicos consultando bases de datos
Compliance / QA automatizado	"¿este texto cumple la política X?", verificación de campos obligatorios



GEPA → DSPy

GEPA optimiza el prompt.

¿Pero quién diseña el sistema?

GEPA optimiza

El texto de las instrucciones – y solo eso.

GEPA no toca

Qué entra y sale. Cómo razona el modelo.
Quién define la tarea.

DSPy agrega

Declarar el sistema completo: tarea, estrategia y prompt – y luego optimizarlo.

Conceptos clave de DSPy

Signature – qué entra y qué sale

Metric – cómo medir si la respuesta es buena

Module – estilo de razonamiento

Optimizer – afina el prompt solo

Program – pasos encadenados

LM – el motor LLM (GPT, Claude...)